LSTM

W zadaniach NLP (Natural Language Processing), takich jak:

- analiza sentymentu,
- tłumaczenie maszynowe,
- generowanie tekstu,
- predykcja kolejnego słowa, potrzebujemy modeli, które rozumieją sekwencję słów, a nie tylko pojedyncze słowa.

Tu właśnie używamy **LSTM** — sieci, która potrafi pamiętać kontekst w czasie (np. zależności gramatyczne lub semantyczne).

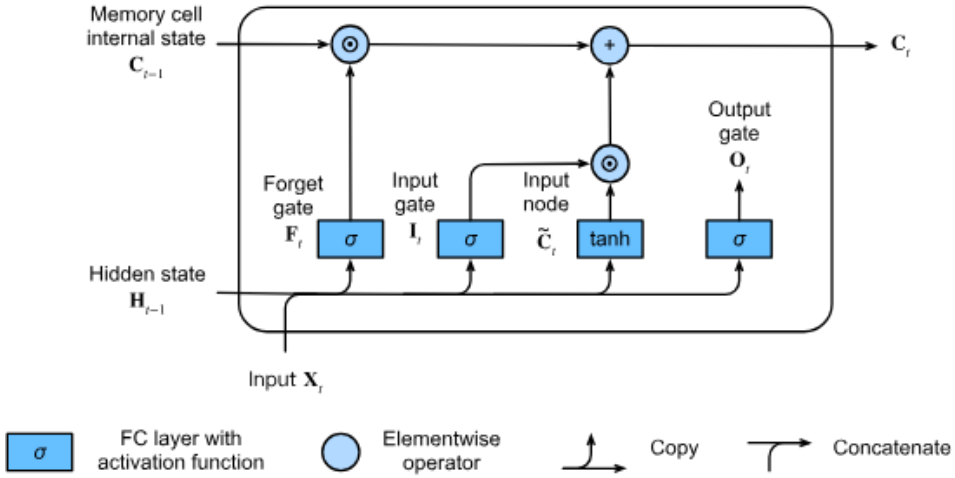
Kiedy używać LSTM?

- gdy sekwencje mają od umiarkowanej do dużej długości i **zależności czasowe są ważne**
- gdy mamy mało danych i nie chcemy używać cięższych modeli (np. transformery - to już niedługo :)), albo chcemy modelu, który działa dobrze na mniejszych zbiorach
- do zadań sekwencyjnych

Kiedy nie używać?

- gdy mamy ogromne zbiory i potrzebujemy najlepszych wyników — transformery (np. BERT) często biją LSTM
- gdy zadanie wymaga globalnych zależności i kontekstów, które lepiej łapią mechanizmy self-attention

Jak zbudowany jest LSTM?



Podstawowy blok LSTM składa się z: stanu komórkowego (cell state) i wewnętrznych bramek:

- **forget gate** (bramka zapominania) — decyduje, które informacje z poprzedniej komórki zignorować
- **input gate** (bramka wejścia) — decyduje, które nowe informacje dodać
- **output gate** (bramka wyjścia) — decyduje, jaka część stanu komórkowego stanie się wyjściem

Przykład - przewidywanie kolejnego słowa w zdaniu

```
!pip install flair
```

```
Requirement already satisfied: six in /usr/local/lib/python3.13/dist-packages (from langdetect>=1.0.9->flair) (1.17.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=2.2.3->flair) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=2.2.3->flair) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=2.2.3->flair) (4.67.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=2.2.3->flair) (1.5.1)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=2.2.3->flair) (2.1.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=2.2.3->flair) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=2.2.3->flair) (3.3.3)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from mpld3>=0.3->flair) (3.1.6)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn>=1.0.2->flair) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn>=1.0.2->flair) (1.6.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn>=1.0.2->flair) (3.7.0)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.13/dist-packages (from torch>=1.13.1->flair) (80.10.2)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.13/dist-packages (from torch>=1.13.1->flair) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.13/dist-packages (from torch>=1.13.1->flair) (3.7)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.13/dist-packages (from transformers<5.0.0,>=4.25.0->transformers[s
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.13/dist-packages (from transformers<5.0.0,>=4.25.0->transformers[sentencepi
Requirement already satisfied: sentencepiece!=0.1.92,>=0.1.91 in /usr/local/lib/python3.13/dist-packages (from transformers[sentencepiece]<5.0.0,>=4.25
Requirement already satisfied: protobuf in /usr/local/lib/python3.13/dist-packages (from transformers[sentencepiece]<5.0.0,>=4.25.0->flair) (6.33.6)
Requirement already satisfied: click==8.5.0 in /usr/local/lib/python3.13/dist-packages (from wikipedia-api>=0.5.7->flair) (8.5.0)
Requirement already satisfied: httpx==0.28.1 in /usr/local/lib/python3.13/dist-packages (from wikipedia-api>=0.5.7->flair) (0.28.1)
Requirement already satisfied: tenacity==9.1.4 in /usr/local/lib/python3.13/dist-packages (from wikipedia-api>=0.5.7->flair) (9.1.4)
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx==0.28.1->wikipedia-api>=0.5.7->flair) (4.15.1)
Requirement already satisfied: certifi in /usr/local/lib/python3.13/dist-packages (from httpx==0.28.1->wikipedia-api>=0.5.7->flair) (2026.7.22)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages (from httpx==0.28.1->wikipedia-api>=0.5.7->flair) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.13/dist-packages (from httpx==0.28.1->wikipedia-api>=0.5.7->flair) (3.20)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->httpx==0.28.1->wikipedia-api>=0.5.7->flair) (0
Requirement already satisfied: urllib3!=2.2.0,<3,>=1.25.4 in /usr/local/lib/python3.13/dist-packages (from botocore<1.44.0,>=1.43.117->boto3>=1.20.27-
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from joblib>=1.2.0->scikit-learn>=1.0.2->flair) (3.1.2)
Requirement already satisfied: attrs>=19.2.0 in /usr/local/lib/python3.13/dist-packages (from jsonlines>=1.2.0->bioc<3.0.0,>=2.0.0->flair) (26.1.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from sympy>=1.13.3->torch>=1.13.1->flair) (1.3.0)
Requirement already satisfied: accelerate>=0.26.0 in /usr/local/lib/python3.13/dist-packages (from transformers[sentencepiece,torch]<5.0,>=4.1->transfo
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.13/dist-packages (from beautifulsoup4->gdown>=4.4.0->flair) (3.0.3)
Requirement already satisfied: sortedcontainers in /usr/local/lib/python3.13/dist-packages (from intervaltree->bioc<3.0.0,>=2.0.0->flair) (2.4.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from jinja2->mpld3>=0.3->flair) (3.0.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests[socks]->gdown>=4.4.0->flair) (3.4.9)
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/python3.13/dist-packages (from requests[socks]->gdown>=4.4.0->flair) (1.7.1)
Requirement already satisfied: psutil in /usr/local/lib/python3.13/dist-packages (from accelerate>=0.26.0->transformers[sentencepiece,torch]<5.0,>=4.1
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import numpy as np
import random

from flair.data import Sentence
from flair.embeddings import FlairEmbeddings
```

```
SEED = 42
random.seed(SEED)
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x7efd4503cc70>
```

Tworzymy słownik

```
sentences = [
  "It was a cold morning in October.",
  "Anna stood by the window, watching the fog rise over the city.",
  "She wrapped her scarf tighter and checked her phone.",
  "A message from Tom appeared: 'Are you ready for today?'.",
  "She smiled, remembering their plan.",
  "They were going to meet at the train station at dawn.",
  "The streets were silent, except for the sound of her footsteps.",
  "As she turned the corner, she saw a figure waiting in the mist.",
  "It was him."
]

# Inicjalizacja embeddingów
embedding = FlairEmbeddings('news-forward')

# dla każdego zdania tworzymy obiekty Sentence
flair_sentences = [Sentence(s) for s in sentences]

# Wydobycie embeddingów słów
for s in flair_sentences:
    embedding.embed(s)

# Zbudowanie słownika słów na podstawie korpusu
words = set(" ".join(sentences).split())
word2idx = {word: i+1 for i, word in enumerate(words)}  # indeksy od 1 (0 = padding)
idx2word = {i: word for word, i in word2idx.items()}
vocab_size = len(word2idx) + 1

print("Słownik:", word2idx)
print("Rozmiar słownika:", vocab_size)
```

```
Słownik: {'train': 1, 'Tom': 2, 'saw': 3, 'a': 4, 'by': 5, 'tighter': 6, 'him.': 7, 'morning': 8, 'streets': 9, 'she': 10, 'for': 11, 'mist.': 12, 'thei
Rozmiar słownika: 65
```

Każde słowo dostaje unikalny numer.

Indeks 0 zarezerwowany na padding (wyrównywanie długości sekwencji).

Przewidywanie kolejnego słowa

Będzi trenować lstm w kontekście całej sekwencji, żeby przewidywał nam kolejne słowo. Np: "To był chłodny" -> "poranek"

Tworzymy dataset na podstawie nazszych zdań

```
import torch
import torch.nn as nn
import torch.optim as optim


all_words = " ".join(sentences).split() # wszystkie słowa (to co mamy w słowniku)
seq_length = 4 # będziemy przewidywać piąte słowo na podstawie 4 poprzednich


X_data, y_data = [], []
# sztucznie tworzymy przykłady
for i in range(len(all_words) - seq_length):
  # przesuwamy nasze okienko sekwencji słów po tekście
  X_seq = all_words[i:i+seq_length]
  y_seq = all_words[i+seq_length]
  # nie używamy słów tylko index ze slownika
  X_data.append([word2idx[w] for w in X_seq])
  y_data.append(word2idx[y_seq])


# robimy tensory na których będziemy trenować
X = torch.tensor(X_data)
y = torch.tensor(y_data)


print(f"Sample input: {X_data[0]} -> target: {y_data[0]}")
```

```
Sample input: [25, 15, 4, 54] -> target: 8
```

Na co patrzymy?

Przygotowaliśmy sobie mechanizm że model dostaje 4 słowa (o indeksach 45, 20, 13, 17), a jego zadaniem jest przewidzieć kolejne słowo (o indeksie 5).

Stwórzmy model

Stwórzmy model który od razu zrobi to co ręcznie zrobiliśmy u góry

```
class WordPredictorLSTM(nn.Module):
  def __init__(self, vocab_size, embedding_dim, hidden_dim):
    super().__init__()
    # to nasz embeding modelu - go też będziemy trenować!
    # każde słowo zamieniamy na wektor o długości embedding_dim
    self.embedding = nn.Embedding(vocab_size, embedding_dim)
    # tu jest nasz model :)
    self.lstm = nn.LSTM(embedding_dim, hidden_dim, batch_first=True)
    # mapujemy wynik LSTM na rozkład prawdopodobieństwa z wszystkich słów w słowniku
    self.fc = nn.Linear(hidden_dim, vocab_size)


  def forward(self, x):
    # x - przyszło do nas słowo (index w słowniku)
    # zmieniamy index na wektor
    embeds = self.embedding(x)
    # przepuszczamy przez lstm
    # out: ukryte stany dla każdego kroku czasowego,
    # h: ostatni ukryty stan (zawiera podsumowanie sekwencji!) - Ważne !,
    # c: wewnętrzny stan pamięci.
    out, (h, c) = self.lstm(embeds)
    # ostatni ukryty stan, zwracamy prawdopodobieństwa - które słowo ma najwyzsze prawdopodobieństwo?
    return self.fc(h[-1])


model_word = WordPredictorLSTM(vocab_size=vocab_size, embedding_dim=100, hidden_dim=128)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model_word.parameters(), lr=0.001)
```

Trening

```
EPOCHS = 200
for epoch in range(EPOCHS):
  optimizer.zero_grad() # czyścimy gradienty
  output = model_word(X) # przewidywania modelu
  loss = criterion(output, y) # porównujemy przewidywanie vs rzeczywiste słowo
  loss.backward() # liczymy gradienty
  optimizer.step() # aktualizujemy wagi
```

```
  if (epoch+1) % 40 == 0:
    print(f"Epoch {epoch+1}/{EPOCHS}, Loss: {loss.item():.4f}")
```

```
Epoch 40/200, Loss: 1.5429
Epoch 80/200, Loss: 0.0877
Epoch 120/200, Loss: 0.0281
Epoch 160/200, Loss: 0.0159
Epoch 200/200, Loss: 0.0105
```

Przetestujmy nasz model! 😼

```
def generate_text(model, seed_text, next_words=8):
  model.eval() # nie aktualizuj wag
  words = seed_text.split() # pierwsze zdanie zamieniamy na listę słów
  for _ in range(next_words): # generujemy tyle kolejnych słów ile chcemy
    seq = [word2idx.get(w, 0) for w in words[-seq_length:]] # zmieniamy na indeksy nasz kontekst, seq_length zdefiniowaliśmy podczas tworzenia datasetu
    x = torch.tensor(seq).unsqueeze(0) # tensor o wymiarze (1, długość kontekstu)
    with torch.no_grad():
      output = model(x) # przewidujemy na podstawie otrzymanych 4 słów
      predicted_idx = torch.argmax(output, dim=1).item() # wybieramy jakie słowo ma największe prawdopodobieństwo
      predicted_word = idx2word.get(predicted_idx, '') # indext wybranego słowa -> słowo
      words.append(predicted_word)
  return ' '.join(words) # zwracamy wszystko po kolei (nasza historia)


seed = "It was a cold"
print("Generated continuation:")
print(generate_text(model_word, seed))
```

```
Generated continuation:
It was a cold morning in October. Anna stood by the window,
```

Zadanie

3 pkt

Stwórz sieć LSTM, która przewidzi kolejne zdanie a nie kolejne słowo

Input sentence ⟶ Encoder LSTM ⟶ Hidden state ⟶ Decoder LSTM ⟶ Predicted next sentence

UWAGA! Zproponowano szkielet LSTM SeqToSeq - ale jeśli chcesz zrobić to w inny sposób, użyć innych emebedingów itp - śmiało :)

```
!pip install "flair[word-embeddings]"
Requirement already satisfied: transformers<5.0.0,>=4.25.0 in /usr/local/lib/python3.13/dist-packages (from transformers[sentencepiece]<5.0.0,>=4.25.0
Requirement already satisfied: wikipedia-api>=0.5.7 in /usr/local/lib/python3.13/dist-packages (from flair[word-embeddings]) (0.16.0)
Requirement already satisfied: bioc<3.0.0,>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from flair[word-embeddings]) (2.1)
Collecting gensim>=4.2.0 (from flair[word-embeddings])
  Downloading gensim-4.4.0-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Collecting bpemb>=0.3.5 (from flair[word-embeddings])
  Downloading bpemb-0.3.6-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: jsonlines>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from bioc<3.0.0,>=2.0.0->flair[word-embeddings]) (4.0.0)
Requirement already satisfied: intervaltree in /usr/local/lib/python3.13/dist-packages (from bioc<3.0.0,>=2.0.0->flair[word-embeddings]) (3.2.1)
Requirement already satisfied: docopt in /usr/local/lib/python3.13/dist-packages (from bioc<3.0.0,>=2.0.0->flair[word-embeddings]) (0.6.2)
Requirement already satisfied: botocore<1.44.0,>=1.43.117 in /usr/local/lib/python3.13/dist-packages (from boto3>=1.20.27->flair[word-embeddings]) (1.4
Requirement already satisfied: jmespath<2.0.0,>=0.7.1 in /usr/local/lib/python3.13/dist-packages (from boto3>=1.20.27->flair[word-embeddings]) (1.1.0)
Requirement already satisfied: s3transfer<0.20.0,>=0.19.0 in /usr/local/lib/python3.13/dist-packages (from boto3>=1.20.27->flair[word-embeddings]) (0.1
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (from bpemb>=0.3.5->flair[word-embeddings]) (2.1.3)
Requirement already satisfied: requests in /usr/local/lib/python3.13/dist-packages (from bpemb>=0.3.5->flair[word-embeddings]) (2.32.4)
Requirement already satisfied: sentencepiece in /usr/local/lib/python3.13/dist-packages (from bpemb>=0.3.5->flair[word-embeddings]) (0.2.2)
Requirement already satisfied: wrapt<3,>=1.16 in /usr/local/lib/python3.13/dist-packages (from deprecated>=1.2.13->flair[word-embeddings]) (2.5.1)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.13/dist-packages (from ftfy>=6.1.0->flair[word-embeddings]) (0.9.2)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.13/dist-packages (from gdown>=4.4.0->flair[word-embeddings]) (4.13.5)
Requirement already satisfied: filelock in /usr/local/lib/python3.13/dist-packages (from gdown>=4.4.0->flair[word-embeddings]) (4.2.1)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.13/dist-packages (from gensim>=4.2.0->flair[word-embeddings]) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.13/dist-packages (from gensim>=4.2.0->flair[word-embeddings]) (8.0.4)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub>=0.10.0->flair[word-embeddings]) (202
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub>=0.10.0->flair[word-embeddings])
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub>=0.10.0->flair[word-embeddings]) (26.3
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub>=0.10.0->flair[word-embeddings]) (6.0.3)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub>=0.10.0->flair[word-embeddi
```

```
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->httpx==0.28.1->wikipedia-api>=0.5.7->flair[wor
Requirement already satisfied: urllib3!=2.2.0,<3,>=1.25.4 in /usr/local/lib/python3.13/dist-packages (from botocore<1.44.0,>=1.43.117->boto3>=1.20.27-
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from joblib>=1.2.0->scikit-learn>=1.0.2->flair[word-embedd
Requirement already satisfied: attrs>=19.2.0 in /usr/local/lib/python3.13/dist-packages (from jsonlines>=1.2.0->bioc<3.0.0,>=2.0.0->flair[word-embeddi
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from sympy>=1.13.3->torch>=1.13.1->flair[word-embeddings
Requirement already satisfied: accelerate>=0.26.0 in /usr/local/lib/python3.13/dist-packages (from transformers[sentencepiece,torch]<5.0,>=4.1->transfo
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.13/dist-packages (from beautifulsoup4->gdown>=4.4.0->flair[word-embeddings]) (3
```

```
from flair.data import Sentence
from flair.embeddings import WordEmbeddings, DocumentPoolEmbeddings
import torch.nn.functional as F


sentences = [
    "It was a bright and sunny morning.",
    "The birds chirped happily in the trees.",
    "A cool breeze swept through the streets.",
    "Children played joyfully in the park.",
    "The cat lounged lazily on the windowsill.",
    "The coffee shop smelled of freshly brewed espresso.",
    "Raindrops pattered softly against the roof.",
    "The sky turned shades of pink and orange at sunset.",
    "A dog barked loudly at a passing car.",
    "The leaves rustled under the feet of walkers.",
    "Smoke rose from the chimney into the crisp air.",
    "The waves crashed rhythmically onto the shore.",
    "A lone cyclist rode along the quiet road.",
    "Flowers bloomed vibrantly in the garden.",
    "The old bookstore had a comforting aroma of paper.",
    "Fog rolled over the hills in the early morning.",
    "A gentle hum of traffic filled the city streets.",
    "The clock struck noon with a resounding chime.",
    "Windows rattled in the wind of the approaching storm.",
    "The bakery displayed an array of fresh pastries.",
    "A distant train whistle echoed through the valley.",
    "Stars twinkled faintly in the darkening sky.",
    "The playground was empty as dusk approached.",
    "A fisherman cast his line into the calm river.",
    "The scent of pine trees lingered in the air.",
    "Clouds gathered ominously over the mountains.",
    "The market buzzed with people and chatter.",
    "A child laughed as a balloon floated away.",
    "The lamp post flickered in the evening gloom.",
    "Shadows stretched long across the empty street.",
    "The fountain gurgled peacefully in the square.",
    "A paper airplane sailed across the classroom.",
    "The dog chased its tail in playful circles.",
    "The train station was crowded with travelers.",
```

```
    "Snowflakes drifted silently from the gray sky.",
    "A painter worked diligently on his canvas.",
    "The wind whispered secrets through the trees.",
    "A street musician played a soulful tune.",
    "The river sparkled under the midday sun.",
    "The fire crackled warmly in the hearth.",
    "A cyclist narrowly avoided a puddle on the road.",
    "The library was silent except for turning pages.",
    "The airplane soared high above the clouds.",
    "A butterfly landed delicately on a flower.",
    "The aroma of freshly baked bread filled the kitchen.",
    "The old clock ticked steadily in the hallway.",
    "A cat chased a shadow across the wall.",
    "The garden gate creaked in the evening breeze.",
    "The moon reflected off the still lake.",
    "A boy flew a kite on the open field.",
    "The streets glistened after a sudden rain shower."
]
```

```
# EMBEDINGI
from flair.embeddings import WordEmbeddings, DocumentPoolEmbeddings

flair_sentences = [Sentence(s) for s in sentences]

glove_embedding = WordEmbeddings('en-glove')
document_embedding = DocumentPoolEmbeddings([glove_embedding])

document_embedding.embed(flair_sentences)

print("Embedding pierwszego zdania:", flair_sentences[0].embedding.shape)

embeddings_tensor = torch.stack([s.embedding for s in flair_sentences])

# DATASET - sequence to sequence
X_seq = embeddings_tensor[:-1].unsqueeze(1)
y_seq = embeddings_tensor[1:].unsqueeze(1)

# jeśli tu jest zero to prawdopodobnie coś poszło nie tak :)
print("X_seq wymiar:", X_seq.shape)

# MODEL
class Seq2SeqLSTM(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.encoder = nn.LSTM(input_dim, hidden_dim, batch_first=True)
        self.decoder = nn.LSTM(input_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, input_dim)
```

```
    def forward(self, x):
        # encoder
        _, (hidden, cell) = self.encoder(x)

        # decoder
        decoder_input = torch.zeros_like(x)
        decoder_out, _ = self.decoder(decoder_input, (hidden, cell))

        # return
        predicted = self.fc(decoder_out)
        return predicted

input_dim = X_seq.shape[2]
hidden_dim = 64
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model_seq = Seq2SeqLSTM(input_dim, hidden_dim).to(device)

X_seq = X_seq.to(device)
y_seq = y_seq.to(device)

# Trening
criterion = nn.MSELoss()
optimizer = optim.Adam(model_seq.parameters(), lr=0.011)

EPOCHS = 300
model_seq.train()
for epoch in range(EPOCHS):
    optimizer.zero_grad()
    predictions = model_seq(X_seq)
    loss = criterion(predictions, y_seq)
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 50 == 0:
        print(f"Epoch {epoch+1}/{EPOCHS}, Loss: {loss.item():.6f}")
```

```
Embedding pierwszego zdania: torch.Size([100])
X_seq wymiar: torch.Size([50, 1, 100])
Epoch 50/300, Loss: 0.018045
Epoch 100/300, Loss: 0.007355
Epoch 150/300, Loss: 0.002120
Epoch 200/300, Loss: 0.000639
Epoch 250/300, Loss: 0.000213
Epoch 300/300, Loss: 0.000421
```

```
import torch
import torch.nn.functional as F

device = next(model_seq.parameters()).device

test_idx = 1
X_input = flair_sentences[test_idx].embedding.to(device).unsqueeze(0).unsqueeze(0)

# Model w trybie ewaluacyjnym
model_seq.eval()
with torch.no_grad():
    predicted_embedding = model_seq(X_input)

print("Wymiary przewidzianego embedingu", predicted_embedding.shape)

# Zamieniamy na 1D tensor
pred_emb = predicted_embedding.squeeze(0).squeeze(0)

# Tworzymy listę embeddingów wszystkich zdań w zbiorze
all_embeddings = embeddings_tensor.to(device)
similarities = F.cosine_similarity(pred_emb.unsqueeze(0), all_embeddings, dim=1)

best_idx = torch.argmax(similarities).item()

print("Zdanie wejściowe: ", sentences[test_idx])
print("Oczekiwane kolejne zdanie:", sentences[test_idx + 1])
print("Przewidziane kolejne zdanie:", sentences[best_idx])
```

```
Wymiary przewidzianego embedingu torch.Size([1, 1, 100])
Zdanie wejściowe:  The birds chirped happily in the trees.
Oczekiwane kolejne zdanie: A cool breeze swept through the streets.
Przewidziane kolejne zdanie: A cool breeze swept through the streets.
```